

Adam Rassem

Michał Kościanek

Kościuszkon 2026

Wykrywanie Spoofingu sygnału GPS

1. Założenia

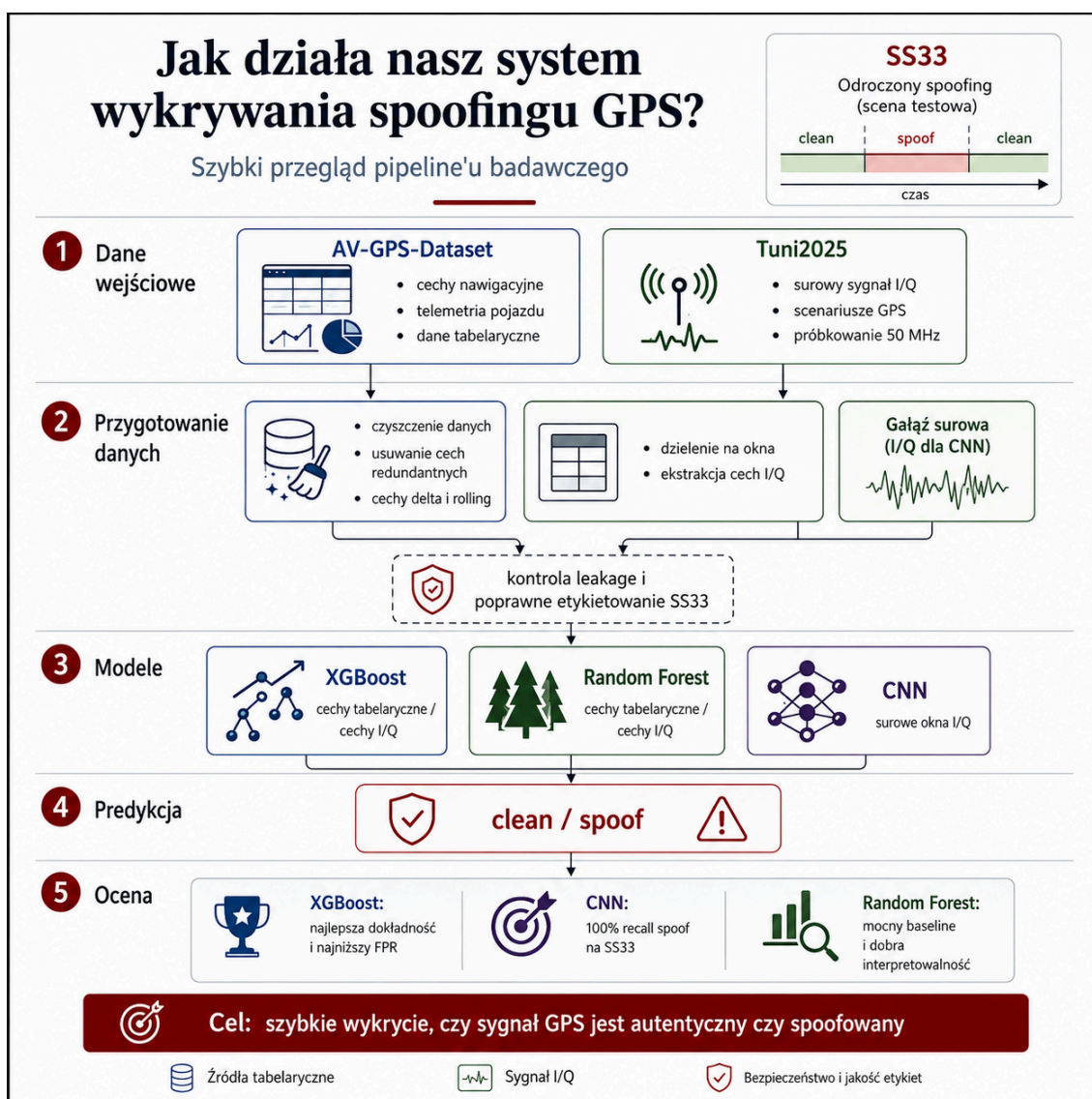
1.1. Dane

W ramach naszego eksperymentu wybraliśmy dwa zbiory danych

- AV-GPS-Dataset
- Tuni 2025

1.2. Specyfikacja

- RTX 6000 PRO Blackwell 96GB VRAM
- Xeon® 6767P 32/128 Cores
- Pamięć RAM 128 GB DDR5
- 600 GB Dysk NVmE



1.2.1. Omówienie IEEE Tuni2025

Jest to publiczny zestaw danych zawierający surowe próbki sygnałów GNSS w postaci I/Q, przeznaczony do badań nad detekcją spoofingu oraz analizą cech fizycznej warstwy sygnału. Cały dataset obejmuje 17 scenariuszy pomiarowych, z czego 8 dotyczy systemu Galileo E1, a 9 systemu GPS L1. W naszej pracy skupiono się wyłącznie na części GPS, ponieważ pozwala ona zawęzić problem do jednego systemu nawigacyjnego i uprościć etap przygotowania danych oraz trenowania modeli.

Każdy scenariusz trwa około 150 s i jest zapisany jako osobny plik binarny zawierający próbki I/Q z częstotliwością próbkowania 50 MHz. Dane rejestrowano z wykorzystaniem odbiornika SDR USRP NI-2954 oraz anteny NovAtel GNSS-703, natomiast sygnały spoofingowe generowano laboratoryjnie za pomocą generatora Spectracom GSG-6. Oznacza to, że w danych występuje mieszanina rzeczywistych sygnałów satelitarnych odebranych z anteny oraz sygnałów sztucznie wprowadzonych przez spoofer. W części GPS dostępne są zarówno scenariusze czyste, oznaczane jako C, jak i scenariusze spoofingowe, oznaczane jako SS. Scenariusze czyste zawierają sygnały GNSS bez aktywnego ataku i mogą służyć jako klasa referencyjna. Scenariusze spoofingowe obejmują różne warianty ataku: spoofing statyczny bez wielodrogowości, spoofing z symulowaną wielodrogowością oraz scenariusz opóźnionej aktywacji spoofingu, w którym sygnał atakujący pojawia się dopiero w trakcie nagrania. Dla części GPS odpowiadają temu m.in. scenariusze C-5, C-7, SS-17, SS-18, SS-20, SS-27, SS-28, SS-29 oraz SS-33.

Istotną cechą tego zbioru jest to, że w większości scenariuszy spoofowane kody PRN zostały dobrane tak, aby były różne od rzeczywistych PRN widocznych na niebie. Dzięki temu możliwe jest jednoznaczne rozróżnienie sygnałów autentycznych i spoofowanych na poziomie PRN, co jest szczególnie przydatne przy budowie modeli klasyfikacyjnych oraz przy analizie cech sygnału przed korelacją i po korelacji. Wyjątkiem jest scenariusz SS-33, który odpowiada przypadkowi delayed spoofing/meaconing, gdzie spoofowane są wszystkie PRN.

Z punktu widzenia naszej pracy szczególnie ważne jest to, że dataset dostarcza dane niskopoziomowe, czyli nie gotowe pozycje GPS, lecz surowy sygnał radiowy. Pozwala to przygotowywać własne okna czasowe, wyznaczać cechy statystyczne lub widmowe oraz trenować modele uczenia maszynowego bezpośrednio na reprezentacji sygnału. Jednocześnie duży rozmiar plików — około 30 GB na scenariusz — wymaga selektywnego przetwarzania danych, np. przez dzielenie sygnału na krótkie okna i losowanie ograniczonej liczby fragmentów z każdego pliku.

Na potrzeby eksperymentów potraktowano scenariusze czyste jako klasę clean, a scenariusze spoofingowe jako klasę spoofed. Taki podział pozwala sformułować problem jako klasyfikację binarną, której celem jest rozpoznanie, czy dany fragment sygnału pochodzi z warunków normalnej pracy odbiornika, czy z warunków obecności ataku spoofingowego. W dalszej części pracy opisano sposób przygotowania takich fragmentów, proces ekstrakcji danych treningowych oraz zastosowane modele klasyfikacyjne.

pełny opis całego zbioru danych: <https://ieeexplore.ieee.org/document/11222626>

1.2.2. Omówienie AV-GPS-Dataset

AV-GPS-Dataset jest publicznym zbiorem danych przeznaczonym do badań nad detekcją spoofingu GPS. Dane zostały zebrane z wykorzystaniem testowej platformy **ACL-Rover**, czyli pojazdu autonomicznego sterowanego na podstawie nawigacji GPS. Dataset jest powiązany z pracą dotyczącą systemu **GPS-IDS**, czyli anomaly-based framework for GPS spoofing attack detection

W przeciwieństwie do Tuni2025, AV-GPS-Dataset nie zawiera surowych próbek radiowych I/Q. Jest to zbiór tabelaryczny, w którym każdy rekord opisuje stan pojazdu oraz odbiornika GPS za pomocą gotowych cech numerycznych. Dataset obejmuje **44** cechy nawigacyjne i telemetryczne zebrane podczas jazdy pojazdu autonomicznego zarówno w warunkach normalnych, jak i podczas narzuconego ataku GPS spoofing.

Cechy w zbiorze można podzielić na kilka grup. Pierwszą grupę stanowią parametry orientacji i ruchu pojazdu, takie jak **Roll**, **Pitch**, **Heading**, **Yaw**, **Yaw Rate**, **Velocity** oraz **Steering Angle**. Opisują one położenie katowe pojazdu, jego kierunek jazdy, prędkość oraz reakcje układu sterowania. Drugą grupę stanowią cechy pozycyjne i trajektoryjne, m.in. **GPS Latitude**, **GPS Longitude**, **Longitudinal Position**, **Lateral Position**, **Vertical Position**, **Travelled Distance**, **Distance To Home** oraz **X-Track Error**.

Istotną część zbioru stanowią również cechy związane z jakością sygnału GPS i konfiguracją satelitów. Należą do nich m.in. **GPS HDOP**, **GPS VDOP**, **GPS Course**, **Satellite Locks** oraz **Satellite Count**. Parametry **HDOP** i **VDOP** opisują geometryczną jakość rozwiązania GPS, natomiast liczba satelitów i liczba zablokowanych sygnałów pozwalają ocenić stabilność odbioru sygnału nawigacyjnego.

Zbiór zawiera także cechy pomocnicze związane z pracą pojazdu, takie jak **Throttle**, **Temperature**, **Longitudinal Vibration**, **Lateral Vibration** oraz **Vertical Vibration**. Mogą one być użyteczne, ponieważ atak spoofingowy może wpływać nie tylko na samą pozycję GPS, ale również na zachowanie systemu sterowania pojazdem, jego odchylenie od zadanej trasy oraz dynamikę ruchu.

Etykieta klasy znajduje się w kolumnie **Data Type**. Wartość **0** oznacza dane normalne, natomiast wartość **1** oznacza dane z atakiem GPS spoofing. Dzięki temu problem można bezpośrednio sformułować jako klasyfikację binarną: **normal** kontra **spoofing**. Na potrzeby eksperymentów AV-GPS-Dataset wykorzystano głównie do metod klasycznego uczenia maszynowego, takich jak **XGBoost** oraz **Random Forest**. Ponieważ dane mają postać tabelaryczną, nie wymagają one przetwarzania surowego sygnału radiowego. Kluczowe było natomiast oczyszczenie danych, usunięcie redundantnych kolumn, dodanie cech różnicowych i kroczących oraz wykonanie podziału train/validation/test w taki sposób, aby nie mieszać fragmentów tej samej sekwencji pomiarowej między zbiorami.

1.3. Zastosowane Metody

1.3.1. AV GPS DataSet

1.3.1.1. XGBoosting

Dla zbioru AV-GPS-Dataset zastosowano model **XGBoost**, ponieważ dane miały postać tabelaryczną i zawierały wiele cech opisujących stan odbiornika GPS, ruch pojazdu oraz jakość

sygnału. W przeciwieństwie do CNN, model nie pracował bezpośrednio na surowym sygnale I/Q, lecz na przygotowanych wcześniej cechach numerycznych. W pierwszym etapie wykonano analizę korelacji pomiędzy kolumnami. Część cech była silnie redundantna, np. GPS HDOP i GPS VDOP, Satellite Count i Satellite Locks, różne reprezentacje prędkości oraz cechy opisujące wibracje. Z takich grup pozostawiono jednego reprezentanta, a pozostałe kolumny usunięto, aby ograniczyć wymiar danych i uprościć interpretację modelu. Następnie dodano cechy opisujące lokalną dynamikę sygnału. Dla wybranych kolumn obliczono różnice względem poprzedniej próbki w ramach tego samego Measurement_ID, np. zmiany pozycji, kursu, liczby satelitów, błędów trajektorii oraz parametrów jakości GPS. Dodano również cechy kroczące, takie jak średnia i odchylenie standardowe w krótkich oknach czasowych. Dzięki temu model mógł uwzględniać nie tylko aktualną wartość cechy, ale także jej lokalną zmienność. Podział danych wykonano na poziomie Measurement_ID, a nie pojedynczych wierszy. Było to istotne, ponieważ losowe mieszanie próbek mogłoby spowodować, że bardzo podobne fragmenty tej samej sekwencji trafiłyby jednocześnie do zbioru treningowego i testowego. Dane podzielono w proporcjach 70% / 15% / 15% na zbiory treningowy, walidacyjny i testowy. Model XGBoost trenowano jako klasyfikator binarny rozróżniający klasę clean oraz spoofing. Zastosowano dużą liczbę estymatorów oraz niski współczynnik uczenia, aby model uczył się stopniowo. Dodatkowo użyto regularyzacji, ograniczonej głębokości drzew i częściowego próbkowania cech oraz obserwacji, co zmniejszało ryzyko przeuczenia. Parametr scale_pos_weight wykorzystano do uwzględnienia nierównowagi klas. Najważniejszym wynikiem XGBoost było osiągnięcie bardzo wysokiej skuteczności przy niskiej liczbie fałszywych alarmów. Model uzyskał najwyższą accuracy spośród testowanych metod oraz najniższy FPR, co oznacza, że najlepiej ograniczał przypadki oznaczania sygnału clean jako spoofing. Jednocześnie pojawiła się niewielka liczba false negative, dlatego w zadaniu stricte bezpieczeństwa wynik należy interpretować razem z FNR i recall klasy spoof, a nie wyłącznie przez accuracy.

1.3.1.2. Random Forest

W przypadku AV-GPS-Dataset model Random Forest został wykorzystany jako klasyfikator binarny rozróżniający próbki normalne oraz próbki odpowiadające spoofingowi GPS. Dane wejściowe miały postać tabelaryczną, gdzie każdy wiersz opisywał stan lotu oraz parametry jakości sygnału GPS, m.in. kurs, wysokość, liczbę satelitów, HDOP, prędkości, wibracje oraz błąd śledzenia trasy.

Przed treningiem usunięto kolumny tekstowe, techniczne oraz takie, które mogłyby prowadzić do przecieku informacji, np. identyfikatory pomiarów, czas, datę, pozycję GPS oraz etykietę klasy. Usunięto także część cech silnie skorelowanych, aby ograniczyć redundancję danych. Następnie dodano cechy opisujące dynamikę zmian, czyli różnice delta względem poprzedniej próbki w ramach tego samego Measurement_ID, oraz cechy kroczące, takie jak średnie i odchylenia standardowe z krótkich okien czasowych. Dzięki temu model mógł analizować nie tylko pojedynczy stan systemu, ale również zmiany zachodzące w czasie.

Random Forest działa jako zespół wielu drzew decyzyjnych. Każde drzewo uczy się na losowo wybranej części danych oraz losowo wybranym podzbiorem cech. Pojedyncze drzewo podejmuje decyzję przez serię prostych podziałów, np. czy dana wartość cechy jest większa od ustalonego progu. Wynik całego modelu powstaje przez połączenie decyzji wielu drzew. W naszym przypadku model zwracał prawdopodobieństwo klasy spoof, a ostateczna decyzja zależała od dobrego prognozy decyzyjnego.

Zaletą Random Forest jest odporność na szum, możliwość modelowania nieliniowych zależności oraz brak konieczności skalowania cech. Jest to szczególnie przydatne dla danych telemetrycznych, gdzie różne cechy mają odmienne jednostki i zakresy wartości. Dodatkowo zastosowano ważenie klas, ponieważ przypadków normalnych było więcej niż przypadków spoofingu. Dzięki temu model był mniej skłonny do ignorowania rzadszej, ale ważniejszej klasy ataku.

Próg decyzyjny nie został przyjęty automatycznie jako 0.5, lecz dobrany na zbiorze walidacyjnym. W detekcji spoofingu szczególnie istotne jest ograniczenie false negatives, czyli sytuacji, w których aktywny atak zostaje uznany za normalny sygnał. Dlatego oprócz klasycznej metryki F1 analizowano także recall klasy spoof, FNR oraz FPR. Finalny próg dobrano tak, aby zachować wysoką wykrywalność spoofingu przy akceptowalnej liczbie fałszywych alarmów.

1.3.2. Tuni2025 DataSet

1.3.2.1. XGBoosting

W przypadku zbioru Tuni2025 dane wejściowe miały postać dużych plików binarnych `.bin` zawierających surowe próbki sygnału I/Q. Bezpośrednie trenowanie modelu XGBoost na takich danych nie byłoby praktyczne, ponieważ model drzewiasty nie operuje efektywnie na bardzo długich sekwencjach surowego sygnału. Z tego powodu sygnał został najpierw podzielony na krótkie okna czasowe, a następnie dla każdego okna wyznaczono zestaw cech opisujących podstawowe właściwości sygnału.

W wyniku przetwarzania otrzymano pliki `.parquet`, w których każdy wiersz odpowiadał jednemu oknu sygnału. W tabeli znajdowały się między innymi cechy statystyczne i energetyczne składowych I/Q, takie jak średnie, odchylenia standardowe, korelacja między kanałami I oraz Q, średnia moc, statystyki amplitudy, percentyle mocy oraz współczynnik PAPR. Dzięki temu problem detekcji spoofingu został sprowadzony do klasycznej klasyfikacji tabelarycznej.

Dla modelu XGBoost nie wykorzystywano surowych próbek I/Q, lecz wyłącznie wyekstrahowane cechy. Z danych usunięto kolumny techniczne, takie jak nazwa pliku, nazwa scenariusza, indeks okna oraz czas okna, aby model nie uczył się identyfikować scenariuszy na podstawie metadanych. Jako wejście modelu pozostawiono wyłącznie numeryczne cechy sygnałowe.

Istotnym etapem przygotowania danych była kontrola potencjalnego leakage. Początkowo scenariusze `clean C5` oraz `C7` mogłyby zostać potraktowane jako dwa niezależne nagrania, jednak analiza hashy surowych plików wykazała, że są one bajtowo identyczne. W związku z tym nie potraktowano ich jako osobnych scenariuszy treningowych i testowych. Zamiast tego `clean recording` został podzielony blokowo po czasie na rozłączne fragmenty `train`, `validation` oraz `test`, z dodatkowym buforem czasowym pomiędzy nimi.

Dla scenariuszy spoofingowych zastosowano podział po całych scenariuszach. Oznacza to, że okna pochodzące z tego samego scenariusza nie były mieszane pomiędzy `train`, `validation` oraz `test`. Takie podejście ogranicza ryzyko sytuacji, w której model uczy się specyficznych właściwości konkretnego nagrania, a następnie jest oceniany na bardzo podobnych fragmentach tego samego pliku.

Szczegółnej korekty wymagał scenariusz `SS33`, ponieważ zgodnie z dokumentacją nie zawiera on aktywnego spoofingu przez cały czas trwania nagrania. Spoofing rozpoczyna się dopiero po około 70 s, trwa około 80 s i zostaje usunięty około 150 s. Dlatego dla tego scenariusza przygotowano etykietę czasową `label_time`, w której fragmenty przed aktywacją spoofingu oraz

po jego usunięciu oznaczono jako clean, natomiast przedział aktywnego ataku oznaczono jako spoof. Okna znajdujące się blisko granic czasowych ataku pominięto ze względu na niepewność przejścia.

Po przygotowaniu etykiet oraz podziale danych wykonano dodatkową kontrolę duplikatów pomiędzy zbiorami train, validation i test. W tym celu porównano hashe wierszy cech, aby upewnić się, że identyczne okna nie występują jednocześnie w różnych splitach. Dopiero po tej kontroli dane zostały wykorzystane do treningu modelu XGBoost.

1.3.2.2. Random Forest

Random Forest Dla zbioru Tuni2025 model Random Forest został wykorzystany jako klasyczny baseline działający na cechach wyekstrahowanych z okien sygnału I/Q. W przeciwieństwie do CNN, model nie otrzymywał surowej sekwencji próbek, lecz tabelę cech zapisaną w plikach `.parquet`. Każdy wiersz odpowiadał jednemu oknu sygnału i zawierał statystyki opisujące m.in. składowe I/Q, amplitudę, moc oraz percentyle energii. Przed treningiem zastosowano ten sam kontrolowany podział danych co dla pozostałych metod na Tuni2025. Scenariusz C7 został wykluczony, ponieważ okazał się bajtowo identyczny z C5. Pojedyncze nagranie clean C5 podzielono czasowo na fragment treningowy i walidacyjny, natomiast scenariusz SS33 pozostawiono jako held-out test. Dla SS33 użyto etykiet czasowych, ponieważ spoofing występuje tylko w części nagrania. Z danych wejściowych usunięto kolumny techniczne, takie jak nazwa pliku, scenariusz, indeks okna oraz czas okna, aby model nie uczył się rozpoznawać konkretnych scenariuszy na podstawie metadanych. Do modelu przekazano wyłącznie numeryczne cechy sygnałowe, takie jak `i_std`, `q_std`, `amp_mean`, `amp_std`, `power_mean`, `power_std`, `power_p50`, `power_p90`, `power_p99` oraz `papr`. Random Forest zbudowano jako zespół wielu drzew decyzyjnych. Każde drzewo uczyło się na losowej części danych i losowym podzbiorze cech, a końcowa decyzja wynikała z uśrednienia predykcji całego lasu. Takie podejście jest odporne na szum i pozwala modelować nieliniowe zależności między cechami bez konieczności ich skalowania. Model trenowano jako klasyfikator binarny `clean/spoof`. Zastosowano ważenie klas przez `class_weight="balanced"`, aby ograniczyć wpływ nierównej liczby przykładów obu klas. Jako główny próg decyzyjny przyjęto 0.5.

1.3.2.3. CNN na surowym sygnale I/Q

Na podstawie surowych plików `.bin` ze zbioru Tuni2025 przygotowaliśmy pipeline uczenia sieci konwolucyjnej bezpośrednio na sygnale I/Q. Każde nagranie zostało potraktowane jako ciąg próbek zespolonych zapisanych w postaci par I/Q, a następnie podzielone na krótkie okna czasowe. Jedno takie okno stanowiło pojedynczy przykład wejściowy dla modelu CNN.

Dane zostały rozdzielone według scenariuszy, tak aby scenariusz testowy nie występował w treningu. Do treningu wykorzystano scenariusz clean C5 oraz scenariusze spoofingowe SS17, SS18, SS20, SS27 i SS28. Do walidacji wykorzystano wydzielony czasowo fragment nagrania clean oraz scenariusz spoofingowy SS29. Jako główny scenariusz testowy pozostawiono SS33, czyli przypadek opóźnionej aktywacji spoofingu wszystkich widocznych PRN.

W eksperymencie zastosowano ten sam kontrolowany podział danych co dla pozostałych metod na Tuni2025. Scenariusz C7 wykluczono ze względu na identyczność z C5.

Najważniejszą ewaluację przeprowadzono na scenariuszu SS33, który nie był używany podczas treningu. Dla tego scenariusza zastosowano etykiety czasowe zgodne z dokumentacją README: spoofing rozpoczyna się około 70 s po rozpoczęciu nagrania, trwa około 80 s i jest usuwany

około 150 s. Fragmenty przed aktywacją spoofingu oraz po jego zakończeniu oznaczono jako clean, natomiast aktywny przedział spoofingu jako spoof. Niepewne okolice przejść czasowych pominięto.

Na scenariuszu testowym SS33 model CNN osiągnął około 95% accuracy, 100% recall klasy spoof oraz 0% false negative rate. Oznacza to, że model nie przepuścił żadnego aktywnego okna spoofingu jako clean. Błędy występowały głównie jako false positive, czyli fragmenty clean klasyfikowane jako spoofing, szczególnie w pobliżu momentu aktywacji spoofingu.

2. Przebieg Eksperymentu

2.1. Wstępna analiza danych

2.1.1. AV GPS DataSet

2.1.1.1. XGBoosting

Pierwszym etapem pracy z AV-GPS-Dataset była wstępna analiza korelacji oraz redundancji cech. Ponieważ modele drzewiaste, takie jak XGBoost czy Random Forest, są odporne na różne skale zmiennych, nie było konieczne klasyczne skalowanie danych. Istotne było natomiast ograniczenie liczby silnie powtarzających się cech, które niosły niemal tę samą informację jak i pozbycie się duplikatów pomiarów i pustych kolumn.

Na podstawie macierzy korelacji usunięto kolumny należące do grup o bardzo wysokiej współzależności. W każdej takiej grupie pozostawiono jednego reprezentanta, a pozostałe cechy usunięto jako redundantne. Pozwoliło to zmniejszyć wymiar danych, ograniczyć szum informacyjny oraz uprościć interpretację ważności cech w modelu.

Najważniejsze grupy silnie skorelowanych cech były następujące:

- GPS HDOP oraz GPS VDOP,
- Satellite Count oraz Satellite Locks,
- różne reprezentacje prędkości,
- cechy opisujące wibracje,
- cechy pozycji i przebytej drogi.

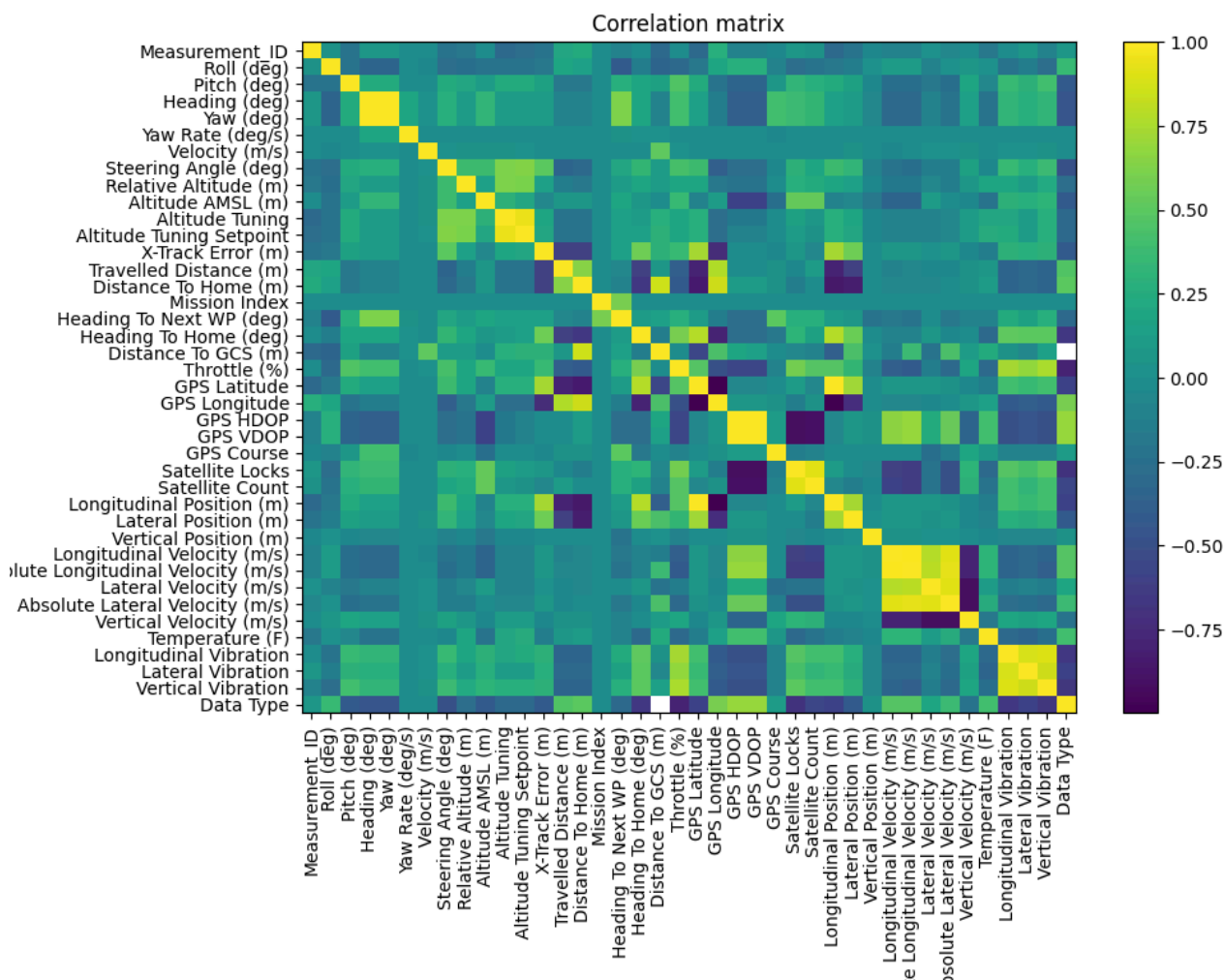
Z każdej grupy pozostawiono cechę najbardziej podstawową albo najłatwiejszą do interpretacji. Przykładowo dla grupy GPS HDOP / GPS VDOP pozostawiono GPS HDOP, natomiast dla grupy Satellite Count / Satellite Locks pozostawiono Satellite Count.

Oprócz oryginalnych wartości cech dodano także cechy różnicowe, oznaczone prefiksem `delta_`. Dla każdej próbki obliczano różnicę względem poprzedniej próbki w ramach tego samego pomiaru `Measurement_ID`.

Takie cechy pozwalają modelowi uwzględnić nie tylko aktualny stan odbiornika, lecz również dynamikę zmian. W kontekście spoofingu jest to istotne, ponieważ atak może nie objawiać się jedynie nietypową wartością pojedynczej zmiennej, ale również nagłą zmianą kursu, pozycji, liczby widocznych satelitów, błędu śledzenia trasy lub parametrów jakości sygnału.

Przykładowo sama wartość GPS HDOP może być jeszcze poprawna, ale jej gwałtowna zmiana względem poprzedniej próbki może wskazywać na niestabilność rozwiązania nawigacyjnego.

Podobnie zmiany GPS Latitude, GPS Longitude, Heading, Yaw Rate lub X-Track Error mogą ujawniać anomalie trudniejsze do wykrycia na podstawie wartości bezwzględnych.



2.1.1.2. Tuni2025 DataSet

W przypadku zbioru Tuni2025 dane wejściowe miały postać dużych plików binarnych `.bin` zawierających surowe próbki sygnału I/Q. Bezpośrednie trenowanie modelu XGBoost na takich danych nie byłoby praktyczne, ponieważ model drzewiasty nie operuje efektywnie na bardzo długich sekwencjach surowego sygnału. Z tego powodu sygnał został najpierw podzielony na krótkie okna czasowe, a następnie dla każdego okna wyznaczono zestaw cech opisujących podstawowe właściwości sygnału.

W wyniku przetwarzania otrzymano pliki `.parquet`, w których każdy wiersz odpowiadał jednemu oknu sygnału. W tabeli znajdowały się między innymi cechy statystyczne i energetyczne składowych I/Q, takie jak średnie, odchylenia standardowe, korelacja między kanałami I oraz Q, średnia moc, statystyki amplitudy, percentyle mocy oraz współczynnik PAPR. Dzięki temu problem detekcji spoofingu został sprowadzony do klasycznej klasyfikacji tabelarycznej.

Dla modelu XGBoost nie wykorzystywano surowych próbek I/Q, lecz wyłącznie wyekstrahowane cechy. Z danych usunięto kolumny techniczne, takie jak nazwa pliku, nazwa scenariusza, indeks okna oraz czas okna, aby model nie uczył się identyfikować scenariuszy na podstawie metadanych. Jako wejście modelu pozostawiono wyłącznie numeryczne cechy sygnałowe.

Istotnym etapem przygotowania danych była kontrola potencjalnego leakage. Początkowo scenariusze clean C5 oraz C7 mogłyby zostać potraktowane jako dwa niezależne nagrania, jednak analiza hashy surowych plików wykazała, że są one bajtowo identyczne. W związku z tym nie potraktowano ich jako osobnych scenariuszy treningowych i testowych. Zamiast tego clean recording został podzielony blokowo po czasie na rozłączne fragmenty train, validation oraz test, z dodatkowym buforem czasowym pomiędzy nimi.

Dla scenariuszy spoofingowych zastosowano podział po całych scenariuszach. Oznacza to, że okna pochodzące z tego samego scenariusza nie były mieszane pomiędzy train, validation oraz test. Takie podejście ogranicza ryzyko sytuacji, w której model uczy się specyficznych właściwości konkretnego nagrania, a następnie jest oceniany na bardzo podobnych fragmentach tego samego pliku.

Szczegółnej korekty wymagał scenariusz SS33, ponieważ zgodnie z dokumentacją nie zawiera on aktywnego spoofingu przez cały czas trwania nagrania. Spoofing rozpoczyna się dopiero po około 70 s, trwa około 80 s i zostaje usunięty około 150 s. Dlatego dla tego scenariusza przygotowano etykietę czasową `label_time`, w której fragmenty przed aktywacją spoofingu oraz po jego usunięciu oznaczono jako clean, natomiast przedział aktywnego ataku oznaczono jako spoof. Okna znajdujące się blisko granic czasowych ataku pominięto ze względu na niepewność przejścia.

Po przygotowaniu etykiet oraz podziale danych wykonano dodatkową kontrolę duplikatów pomiędzy zbiorami train, validation i test. W tym celu porównano hashe wierszy cech, aby upewnić się, że identyczne okna nie występują jednocześnie w różnych splitach. Dopiero po tej kontroli dane zostały wykorzystane do treningu modelu XGBoost.

2.1.2. CNN

2.2. Podział Danych na train/validate/test

2.2.1. Tuni2025

2.2.2. AV GPS DataSet

W przypadku AV-GPS-Dataset dane nie były dzielone losowo po pojedynczych wierszach. Na podstawie kolumny `Run time` utworzono identyfikatory pomiarów `Measurement_ID`, które grupowały kolejne próbki należące do tej samej ciągłej sytuacji pomiarowej. Następnie podział na zbiory treningowy, walidacyjny i testowy wykonano na poziomie całych pomiarów, a nie pojedynczych rekordów.

Takie podejście ogranicza ryzyko przecieku informacji między zbiorami. Gdyby próbki z jednego przejazdu lub jednej sekwencji czasowej zostały losowo rozbite między train i test, model mógłby uczyć się bardzo podobnych fragmentów tej samej sytuacji, a następnie być oceniany na niemal identycznych danych. Wynik takiego eksperymentu byłby zawyżony i nie odzwierciedlałby rzeczywistej zdolności generalizacji modelu.

Po utworzeniu `Measurement_ID` dane podzielono w proporcjach 70% / 15% / 15%:

- 70% pomiarów wykorzystano do treningu modelu,
- 15% pomiarów wykorzystano do walidacji i doboru parametrów,
- 15% pomiarów pozostawiono jako końcowy zbiór testowy.

Zbiór walidacyjny służył do kontroli przeuczenia, doboru hiperparametrów oraz wyboru progu decyzyjnego. Zbiór testowy był używany dopiero na końcu, do oceny finalnego modelu. Dzięki temu wynik testowy lepiej odzwierciedla skuteczność modelu na nowych, niewidzianych wcześniej sekwencjach pomiarowych.

Dodatkowo zachowano proporcje klas clean/spoofing w każdym ze zbiorów, aby model nie był oceniany na zbiorze zawierającym wyłącznie jedną klasę. Było to szczególnie istotne przy metrykach takich jak recall klasy spoof, FNR, FPR, ROC-AUC oraz PR-AUC.

2.3. Dostosowanie parametrów nauczania

2.3.1. AV GPS DataSet

2.3.1.1. XGBoosting

Parametr	Wartość	Znaczenie
objective	binary:logistic	klasyfikacja binarna clean/spoof
eval_metric	aucpr	monitorowanie jakości predykcji probabilistycznych
n_estimators	2000	liczba drzew
learning_rate	0.02	wolniejsze, stabilniejsze uczenie
max_depth	6	ograniczenie złożoności drzew
min_child_weight	1	ograniczenie zbyt drobnych podziałów
gamma	0.0	minimalna poprawa wymagana do podziału
subsample	0.90	uczenie każdego drzewa na części próbek
colsample_bytree	0.9	uczenie każdego drzewa na części cech
reg_lambda	1.0	regularyzacja L2
reg_alpha	0.0	regularyzacja L1
tree_method	hist	szybsza histogramowa metoda budowy drzew
early_stopping_rounds	200	stopowanie nauczania przy braku poprawy

2.3.1.2. Random Forest

W przypadku modelu Random Forest dla AV-GPS-Dataset głównym celem dostrajania było uzyskanie wysokiej wykrywalności klasy spoofing przy możliwie małej liczbie fałszywych alarmów. Model Random Forest składa się z wielu niezależnych drzew decyzyjnych, dlatego najważniejsze parametry dotyczyły liczby drzew, ich maksymalnej złożoności, sposobu losowania cech oraz kompensacji niebalansowania klas.

W pierwszej wersji zastosowano klasyczny wariant Random Forest z dużą liczbą drzew oraz domyślnym progiem decyzyjnym 0.5. Taki model bardzo rzadko oznaczał próbki clean jako spoofing, ale jednocześnie był zbyt zachowawczy i przepuszczał część rzeczywistych ataków. Objawiało się to wysoką precyzją klasy spoof, lecz zbyt niskim recall, czyli dużą liczbą false negative.

Z tego powodu dalsze dostrajanie skupiono nie tylko na parametrach samego lasu, ale również na progu decyzyjnym. Model zwracał prawdopodobieństwo klasy spoofing, dlatego końcowa decyzja nie musiała być podejmowana przy progu 0.5. Na zbiorze walidacyjnym sprawdzono

wiele progów i porównano ich wpływ na **precision**, **recall**, **F1**, **FPR** oraz **FNR**. Ponieważ w detekcji spoofingu szczególnie niebezpieczne są false negatives, preferowano próg dający wysoki recall klasy spoof przy nadal akceptowalnym poziomie false positive.

W finalnej konfiguracji zwiększono liczbę drzew do **900**, aby uśrednienie decyzji było stabilniejsze. Nie ograniczano maksymalnej głębokości drzew, ponieważ dane zawierały złożone zależności między parametrami ruchu, jakością GPS i cechami dynamicznymi. Zastosowano **max_features = "sqrt"**, co oznacza, że każde drzewo przy podziale analizowało tylko losowy podzbiór cech. Dzięki temu drzewa były mniej podobne do siebie, a cały las lepiej generalizował.

Ze względu na nierówny udział klas zastosowano ważenie klas **{0: 1.0, 1: 3.0}**. Klasa spoofing otrzymała większą wagę, aby model silniej karał błędy polegające na pominięciu ataku. Jest to istotne, ponieważ w danych normalne próbki występowały częściej niż próbki spoofingowe, a bez ważenia model mógłby preferować klasę większościową.

Po treningu modelu dobrano dwa progi decyzyjne. Pierwszy próg maksymalizował **F1** na zbiorze walidacyjnym. Drugi wariant był progiem typu high-recall, dobranym tak, aby utrzymać wysoką wykrywalność spoofingu nawet kosztem niewielkiego wzrostu liczby fałszywych alarmów. W praktyce taki próg jest bardziej sensowny dla systemu bezpieczeństwa, ponieważ lepiej wykryć podejrzany przypadek i później go zweryfikować, niż przepuścić aktywny atak jako normalny sygnał.

Parametr	Wartość	Znaczenie
n_estimators	900	liczba drzew w lesie; większa liczba stabilizuje predykcję
max_depth	None	drzewa mogły rosnąć bez sztywnego limitu głębokości
min_samples_split	2	minimalna liczba próbek potrzebna do podziału węzła
min_samples_leaf	1	minimalna liczba próbek w liściu
max_features	sqrt	każde drzewo analizowało losowy podzbiór cech
bootstrap	True	każde drzewo uczone było na losowej próbce danych
class_weight	{0: 1.0, 1: 3.0}	większa kara za błędne pominięcie klasy spoof
threshold	dobrany na validation	próg decyzyjny wybierany na podstawie metryk walidacyjnych

2.3.2. Tuni2025 DataSet

2.3.2.1. XGBoosting

Dla zbioru Tuni2025 model XGBoost został dobrany jako regularizowany klasyfikator binarny działający na cechach statystycznych i energetycznych wyekstrahowanych z okien sygnału I/Q. Model nie operował bezpośrednio na surowym sygnale, lecz na tabeli cech zapisanej w plikach **.parquet**.

W eksperymencie zastosowano funkcję celu **binary:logistic**, ponieważ problem został sformułowany jako klasyfikacja binarna: **clean** albo **spoof**. Jako metrykę monitorowaną podczas

treningu wykorzystano **logloss**, która pozwala oceniać jakość predykcji probabilistycznych. Do trenowania użyto trybu **hist**, czyli histogramowej metody budowania drzew, która jest szybsza i dobrze nadaje się do danych tabelarycznych.

Parametry modelu ustawiono w sposób konserwatywny, aby ograniczyć przeuczenie na konkretnych scenariuszach. Liczbę drzew ustawiono na **1200**, przy niskim współczynniku uczenia **learning_rate = 0.01**. Zastosowano płytkie drzewa o głębokości **maxdepth = 3**, co ogranicza złożoność pojedynczego drzewa i zmniejsza ryzyko zapamiętywania szczegółów scenariuszy. Dodatkowo użyto **min_child_weight = 5** oraz **gamma = 0.05**, aby utrudnić tworzenie podziałów opartych na zbyt małej liczbie próbek lub nieistotnej poprawie funkcji celu.

W celu dalszej regularyzacji zastosowano losowe próbkowanie danych i cech: **subsample = 0.75** oraz **colsample_bytree = 0.75**. Oznacza to, że każde drzewo było uczone tylko na części próbek i części cech, co poprawia odporność modelu na przeuczenie. Dodatkowo ustawiono regularyzację L2 **reg_lambda = 10.0** oraz regularyzację L1 **reg_alpha = 0.1**.

Ze względu na niebalansowaną liczbę przykładów **clean** i **spoof**, parametr **scale_pos_weight** wyznaczono automatycznie jako stosunek liczby przykładów klasy **clean** do liczby przykładów klasy **spoof** w zbiorze treningowym. Pozwoliło to częściowo skompensować nierówny udział klas podczas uczenia.

Model zwracał prawdopodobieństwo klasy **spoof**, dlatego oprócz parametrów samego XGBoost dobrano również próg decyzyjny. **Threshold** był wybierany na zbiorze **validation** tak, aby ograniczyć **FPR**, a jednocześnie minimalizować **FNR**. W kontekście detekcji spoofingu szczególnie istotne było ograniczenie **false negatives**, czyli przypadków, w których aktywny spoofing zostałby błędnie uznany za **clean**.

Parametr	Wartość	Znaczenie
objective	binary:logistic	klasyfikacja binarna clean/spoof
eval_metric	logloss	monitorowanie jakości predykcji probabilistycznych
n_estimators	1200	liczba drzew
learning_rate	0.01	wolniejsze, stabilniejsze uczenie
max_depth	3	ograniczenie złożoności drzew
min_child_weight	5	ograniczenie zbyt drobnych podziałów
gamma	0.05	minimalna poprawa wymagana do podziału
subsample	0.75	uczenie każdego drzewa na części próbek
colsample_bytree	0.75	uczenie każdego drzewa na części cech
reg_lambda	10.0	regularyzacja L2
reg_alpha	0.1	regularyzacja L1
tree_method	hist	szybsza histogramowa metoda budowy drzew
scale_pos_weight	neg / pos	kompensacja niebalansowanych klas

2.3.2.2. CNN

Sygnal w zbiorze **Tuni2025** był próbkowany z częstotliwością **50 MSps**, czyli **50 000 000** próbek zespolonych **I/Q** na sekundę. Oznacza to, że okno o długości **65 536** próbek odpowiada czasowi:

$$\frac{65536}{50000000} \approx 0.00131s = 1.31ms$$

Z kolei okno 131 072 próbek odpowiada:

$$\frac{131072}{50000000} \approx 0.00262s = 2.62ms$$

Okno 65 536 próbek obejmuje więc nieco więcej niż jeden okres kodu GPS C/A, który trwa około 1 ms, natomiast okno 131 072 próbek obejmuje około 2.6 ms sygnału. Dzięki temu sieć otrzymuje większy kontekst czasowy niż w przypadku bardzo krótkich fragmentów, co może ułatwiać wykrywanie bardziej złożonych wzorców w surowym sygnale I/Q.

W trakcie eksperymentów sprawdzono wpływ liczby okien wybieranych z każdego scenariusza. Zwiększanie liczby okien powyżej około 1000 na scenariusz nie dawało wyraźnej poprawy jakości modelu. Wynika to prawdopodobnie z wysokiej korelacji kolejnych fragmentów w obrębie tego samego nagrania. Bardziej istotnym parametrem okazała się długość okna, ponieważ zwiększa ona ilość kontekstu czasowego dostępnego dla modelu CNN.

2.3.3. Random Forest

W przypadku modelu Random Forest dla AV-GPS-Dataset głównym celem dostrajania było uzyskanie wysokiej wykrywalności klasy spoofing przy możliwie małej liczbie fałszywych alarmów. Model Random Forest składa się z wielu niezależnych drzew decyzyjnych, dlatego najważniejsze parametry dotyczyły liczby drzew, ich maksymalnej złożoności, sposobu losowania cech oraz kompensacji niebalansowania klas.

W pierwszej wersji zastosowano klasyczny wariant Random Forest z dużą liczbą drzew oraz domyślnym progiem decyzyjnym 0.5. Taki model bardzo rzadko oznaczał próbki clean jako spoofing, ale jednocześnie był zbyt zachowawczy i przepuszczał część rzeczywistych ataków. Objawiało się to wysoką precyzją klasy spoof, lecz zbyt niskim recall, czyli dużą liczbą false negative.

Z tego powodu dalsze dostrajanie skupiono nie tylko na parametrach samego lasu, ale również na progu decyzyjnym. Model zwracał prawdopodobieństwo klasy spoofing, dlatego końcowa decyzja nie musiała być podejmowana przy progu 0.5. Na zbiorze walidacyjnym sprawdzono wiele progów i porównano ich wpływ na precision, recall, F1, FPR oraz FNR. Ponieważ w detekcji spoofingu szczególnie niebezpieczne są false negatives, preferowano próg dający wysoki recall klasy spoof przy nadal akceptowalnym poziomie false positive.

W finalnej konfiguracji zwiększono liczbę drzew do 900, aby uśrednienie decyzji było stabilniejsze. Nie ograniczano maksymalnej głębokości drzew, ponieważ dane zawierały złożone zależności między parametrami ruchu, jakością GPS i cechami dynamicznymi. Zastosowano `max_features = "sqrt"`, co oznacza, że każde drzewo przy podziale analizowało tylko losowy podzbiór cech. Dzięki temu drzewa były mniej podobne do siebie, a cały las lepiej generalizował.

Ze względu na nierówny udział klas zastosowano ważenie klas {0: 1.0, 1: 3.0}. Klasa spoofing otrzymała większą wagę, aby model silniej karał błędy polegające na pominięciu ataku. Jest to istotne, ponieważ w danych normalne próbki występowały częściej niż próbki spoofingowe, a bez ważenia model mógłby preferować klasę większościową.

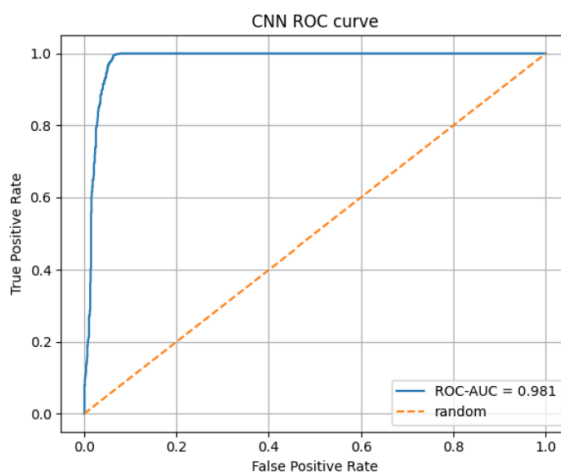
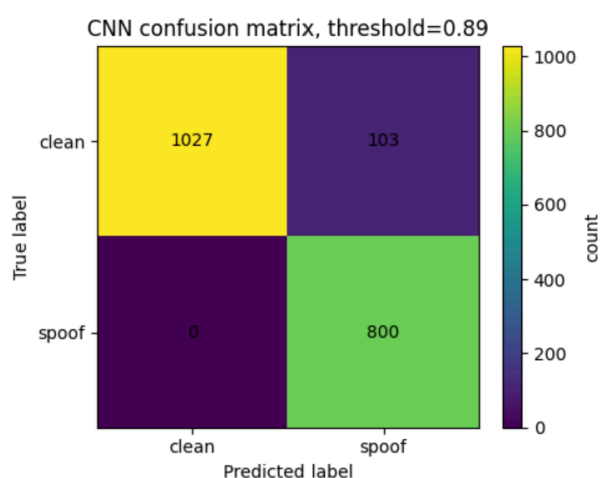
Po treningu modelu dobrano dwa progi decyzyjne. Pierwszy próg maksymalizował F1 na zbiorze walidacyjnym. Drugi wariant był progiem typu high-recall, dobranym tak, aby utrzymać wysoką wykrywalność spoofingu nawet kosztem niewielkiego wzrostu liczby fałszywych alarmów. W praktyce taki próg jest bardziej sensowny dla systemu bezpieczeństwa, ponieważ lepiej wykryć

podejrzany przypadek i później go zweryfikować, niż przepuścić aktywny atak jako normalny sygnał.

Parametr	Wartość	Znaczenie
n_estimators	900	liczba drzew w lesie; większa liczba stabilizuje predykcję
max_depth	None	drzewa mogły rosnąć bez sztywnego limitu głębokości
min_samples_split	2	minimalna liczba próbek potrzebna do podziału węzła
min_samples_leaf	1	minimalna liczba próbek w liściu
max_features	sqrt	każde drzewo analizowało losowy podzbiór cech
bootstrap	True	każde drzewo uczone było na losowej próbce danych
class_weight	{0: 1.0, 1: 3.0}	większa kara za błędne pominięcie klasy spoof
threshold	dobrany na validation	próg decyzyjny wybierany na podstawie metryk walidacyjnych

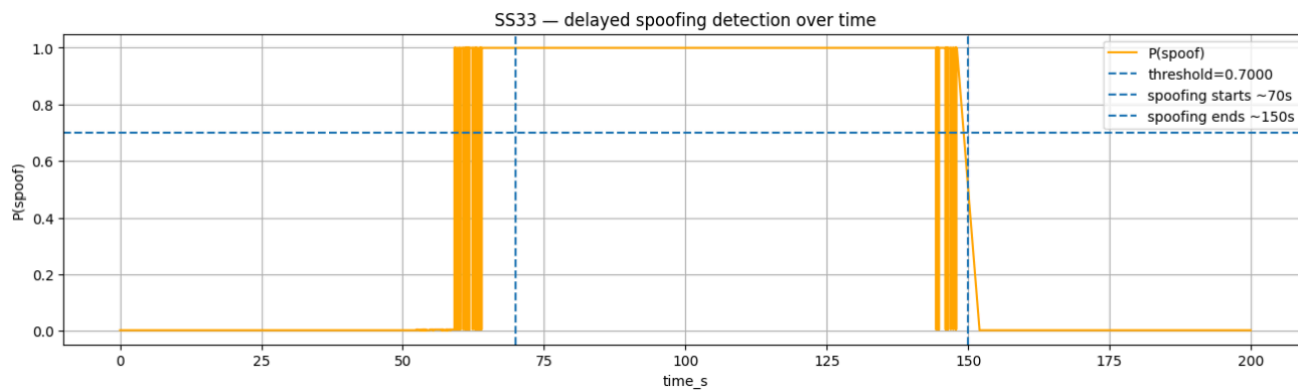
3. Wyniki

3.1. Tuni 2025 CNN

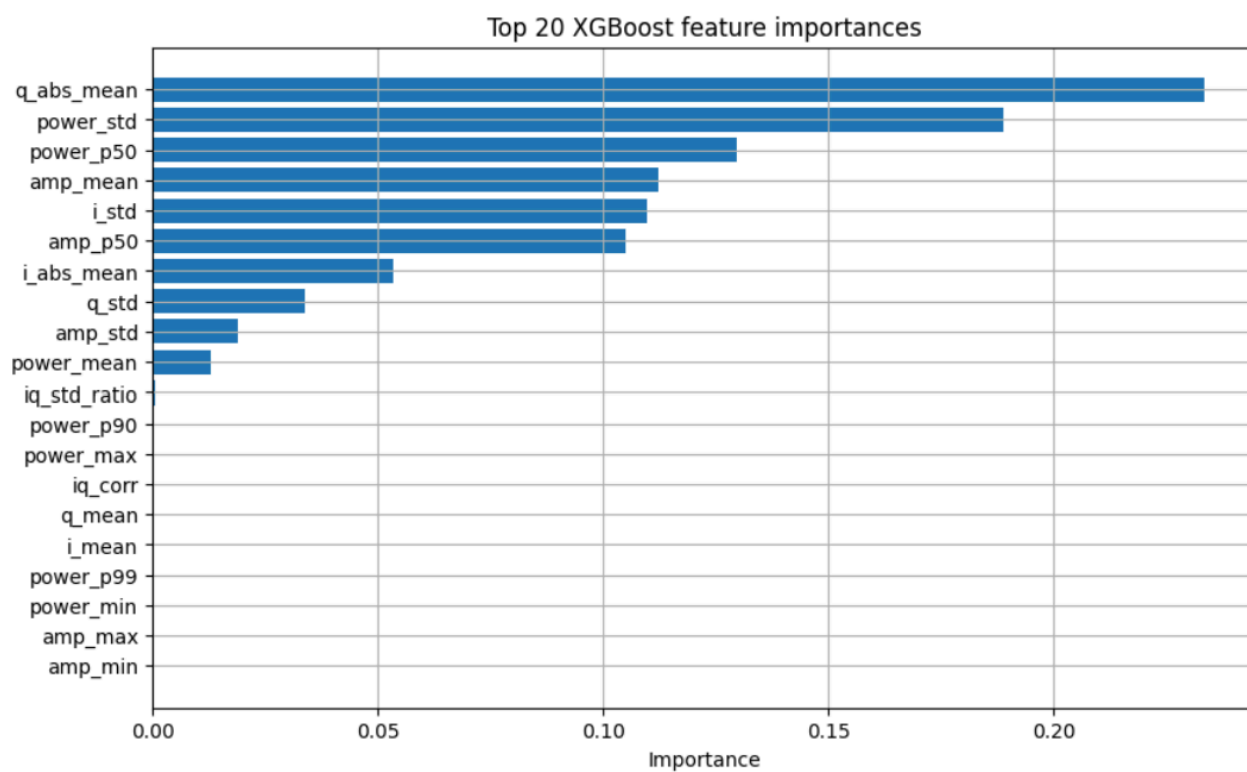


Model	Accuracy	Precision spoof	Recall spoof	F1 spoof	FPR	FNR	ROC-AUC
CNN raw I/Q	94.66%	88.59%	100.00%	93.95%	9.12%	0.00%	0.981

3.2. XGBoost Tuni 2025

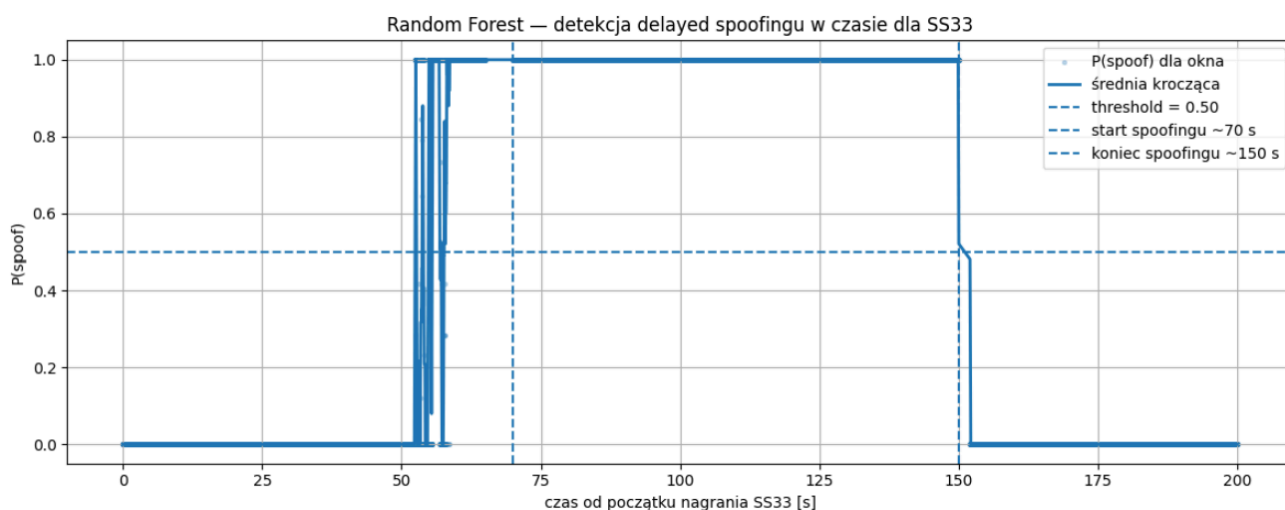
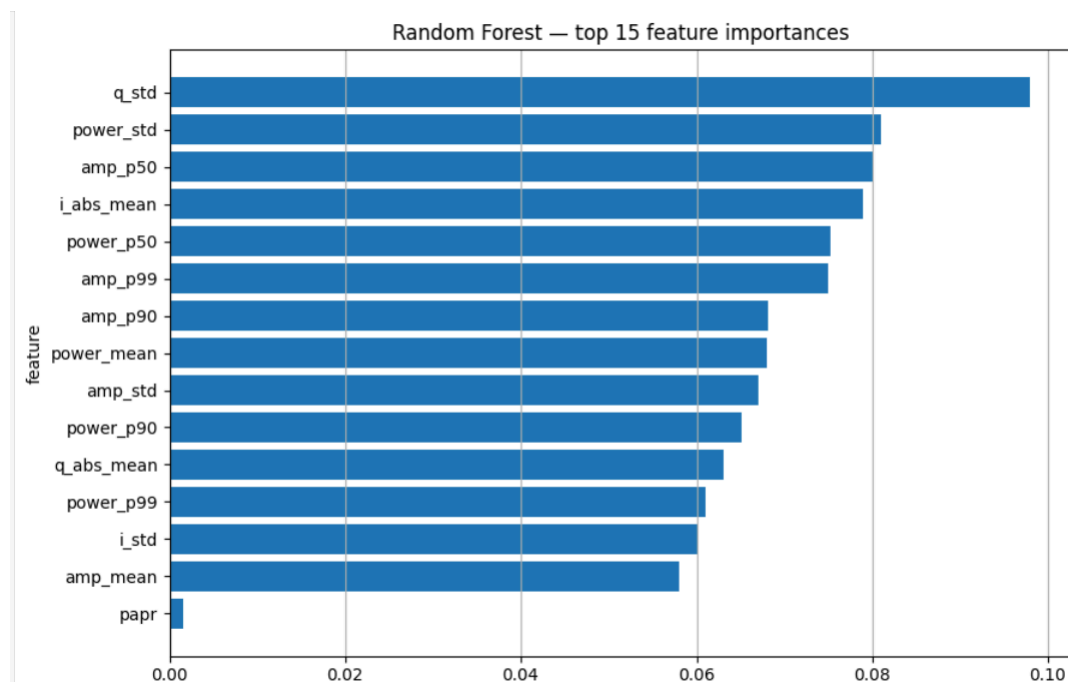


,



Model	Accuracy	Precision spoof	Recall spoof	F1 spoof	FPR	FNR	ROC-AUC	PR-AUC
XGBoost	98.16%	96.99%	99.79%	98.37%	3.86%	0.21%	0.981	0.970

3.3. Random Forest



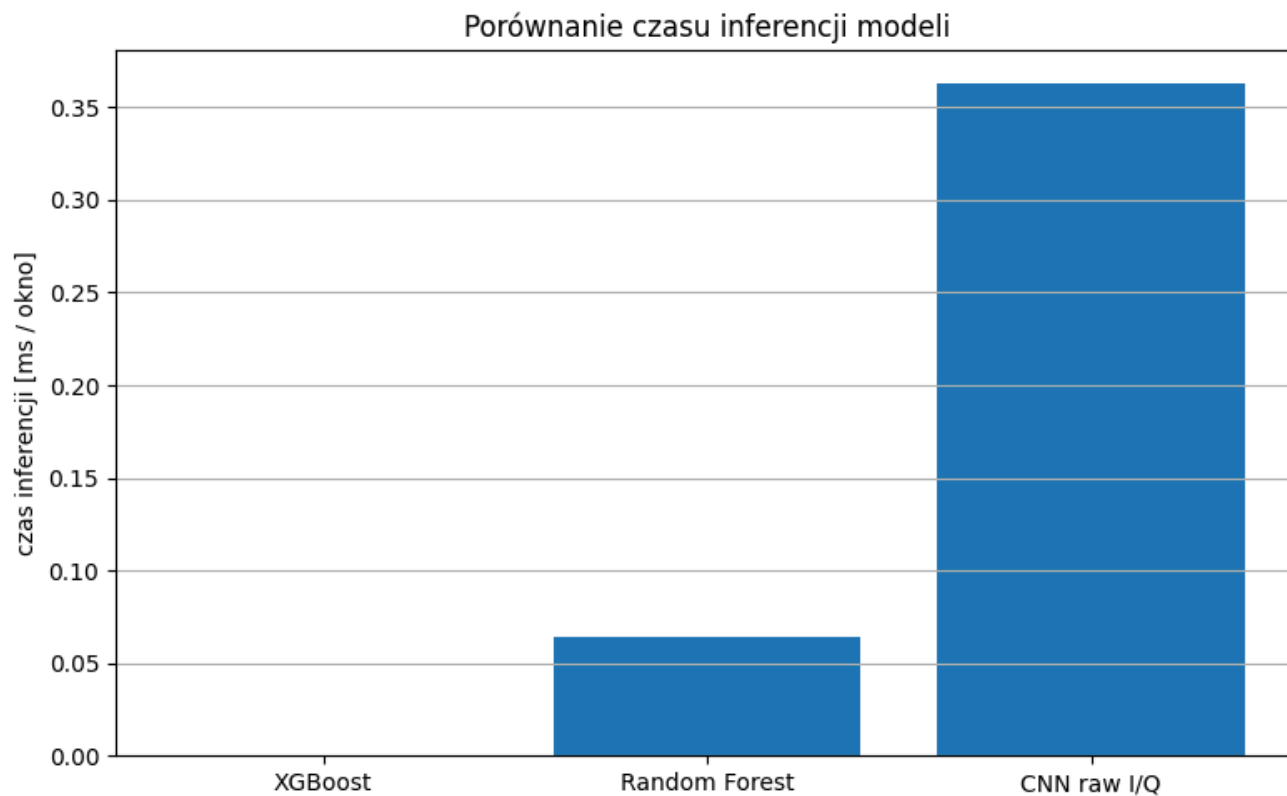
Model	Accuracy	Precision spoof	Recall spoof	F1 spoof	FPR	FNR	ROC-AUC	PR-AUC
Random Forest	94.88%	89.01%	100.00%	94.18%	8.74%	0.00%	0.956	0.890

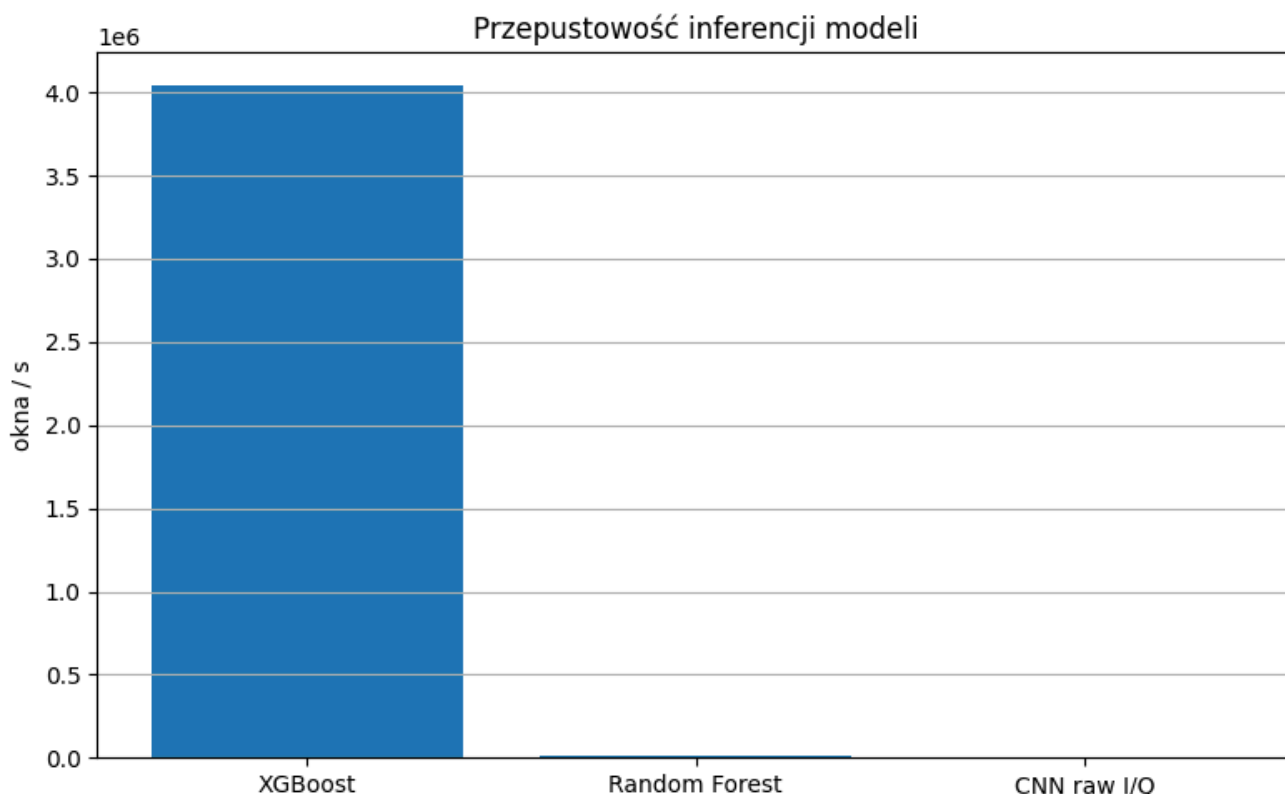
3.3.1. Posumowanie Wyników

Model	Wejście	Accuracy	Precision spoof	Recall spoof	F1 spoof	FPR	FNR	ROC-AUC	PR-AUC
CNN	raw I/Q	94.66%	88.59%	100.00%	93.95%	9.12%	0.00%	0.981	0.951
Random Forest	cechy I/Q	94.88%	89.01%	100.00%	94.18%	8.74%	0.00%	0.956	0.890
XGBoost	cechy I/Q	98.16%	96.99%	99.79%	98.37%	3.86%	0.21%	0.981	0.970

4. Porównanie Wydajności

	model	input	ms_per_window	windows_per_second	realtime_factor_assumed	n_windows
0	XGBoost	cechy I/Q	0.0002	4,042,422	10597.0x	19301
1	Random Forest	cechy I/Q	0.0638	15,669	41.1x	19301
2	CNN raw I/Q	raw I/Q tensor	0.3630	2,755	7.2x	19301





W benchmarku samej inferencji XGBoost i Random Forest były szybsze od CNN, ponieważ operowały na gotowych, krótkich wektorach cech. CNN był wolniejszy, ponieważ przetwarzał bezpośrednio surowe okna I/Q o rozmiarze 2×131072 . Należy jednak zaznaczyć, że pomiar dla RF i XGBoost nie obejmuje kosztu wcześniejszej ekstrakcji cech z surowego sygnału. W praktycznym pipeline pełny czas dla metod drzewiastych obejmowałby etap raw I/Q → cechy, natomiast CNN może działać bezpośrednio na surowych oknach sygnału.

Mimo większego kosztu pojedynczej predykcji CNN osiągnął real-time factor około 7.2x, co oznacza, że inferencja była szybsza od czasu rzeczywistego dla analizowanych okien.

5. Porównanie z literaturą

Bezpośrednie porównanie wyników z innymi pracami dotyczącymi detekcji spoofingu GNSS jest utrudnione, ponieważ autorzy często wykorzystują różne zbiory danych, różne typy ataków, inne poziomy reprezentacji sygnału oraz inne protokoły podziału danych. Z tego powodu poniższe zestawienie należy traktować jako porównanie orientacyjne, a nie jako ścisły ranking metod.

W naszej pracy najważniejszym protokołem testowym był zbiór Tuni2025 GPS, scenariusz SS33 jako held-out test oraz etykiety czasowe dla delayed spoofingu. Scenariusz SS33 nie został oznaczony jedną etykietą dla całego pliku, ponieważ zgodnie z dokumentacją spoofing pojawia się dopiero po około 70 s, trwa około 80 s i kończy się około 150 s. Dzięki temu ewaluacja sprawdzała, czy model potrafi rozróżnić fragmenty clean i spoof w obrębie jednego scenariusza.

Praca / metoda	Dataset / test	Poziom danych	Wynik	Porównalność	Najważniejsze wnioski
Nasza praca: XGBoost	Tuni2025 GPS, SS33 held-out	cechy z okien I/Q	98.16% accuracy ROC-AUC = 0.981 PR-AUC = 0.970 FPR = 3.86% FNR = 0.21%	najbliższy benchmark dla naszego protokołu	najlepszy wynik ogólny i najmniej false positive
Nasza praca: CNN	Tuni2025 GPS, SS33 held-out	surowe okna I/Q	94.66% accuracy 100% recall spoof of FNR = 0% ROC-AUC = 0.981	ten sam test SS33	działa bez ręcznej ekstrakcji cech, bezpośrednio na raw I/Q
Nasza praca: Random Forest	Tuni2025 GPS, SS33 held-out	cechy z okien I/Q	94.88% accuracy 100% recall spoof of FNR = 0% ROC-AUC = 0.956	ten sam test SS33	mocny interpretowalny baseline; ważność cech pokazuje rolę mocy i amplitudy
Rahman, 2025	Tuni2025, wybrane scenariusze	cechy post-correlation / software receiver	83.3% dla SS-1, ok. 79% dla SS-5	częściowo porównywalne: ten sam dataset, inny protokół	niższe wyniki; metoda działa na innym poziomie przetwarzania
Babić et al. 2025	różne scenariusze GNSS	pre-correlation / post-correlation, wavelets + ML	99.99% pre-correlation 87.72% post-correlation	niebezpośrednie: inny dataset i reprezentacja	pokazuje, że pre-correlation zwykle daje przewagę nad post-correlation
Chowdhury, 2025	różne zbiory GNSS	metody ML/statystyczne	90% - 98% accuracy zależnie od zbioru	niebezpośrednie: inne dane	nasz XGBoost mieści się w górnym zakresie wyników literatury
Ghanbarzade i Soleimani, 2025	spoofing/jamming, różne zbiory	ML/DL, computer vision	94% - 98.93% zależnie od modelu	niebezpośrednie: także jamming	wyniki podobnego rzędu, ale dla innego zadania i danych

Zestawienie pokazuje, że bezpośrednie porównanie z literaturą jest trudne, ponieważ prace różnią się zbiorem danych, typem ataku, poziomem reprezentacji sygnału oraz sposobem podziału train/test. Najbliższe porównanie stanowi praca Rahmana, ponieważ również wykorzystuje Tuni2025, jednak stosuje inny poziom przetwarzania sygnału i inne scenariusze testowe.

W naszym protokole najważniejsze było użycie scenariusza SS33 jako held-out testu oraz zastosowanie etykiet czasowych dla delayed spoofingu. Dzięki temu model był oceniany nie

tylko na rozróżnianiu plików clean/spoof, ale na wykrywaniu aktywnego przedziału spoofingu w obrębie jednego nagrania.

Na tym tle wynik XGBoost można traktować jako bardzo mocny punkt odniesienia dla dalszych badań na Tuni2025 GPS. Model uzyskał 98.16% accuracy, ROC-AUC = 0.981, PR-AUC = 0.970 oraz niski FPR = 3.86%. Modele CNN i Random Forest uzyskały z kolei 100% recall klasy spoof oraz FNR = 0%, co oznacza, że w zastosowanym teście nie pominęły aktywnych okien spoofingu.

Najważniejszą różnicą względem wielu wcześniejszych prac jest sposób ewaluacji scenariusza SS33. Zamiast oznaczać cały plik jako spoofingowy, zastosowano etykiety czasowe zgodne z rzeczywistym przedziałem aktywności spoofera. Takie podejście jest bardziej rygorystyczne, ponieważ model musi rozróżnić zarówno fragmenty clean, jak i spoof w obrębie tego samego nagrania.

Na podstawie porównania można stwierdzić, że uzyskany wynik stanowi **bardzo mocny** punkt odniesienia dla dalszych eksperymentów na Tuni2025. Nie jest to jednak bezwzględny dowód **SOTA** dla całej dziedziny GNSS spoofing detection, ponieważ dostępne prace wykorzystują różne datasety i różne protokoły testowe.

6. Posumowanie

W pracy przeanalizowano dwa różne zbiory danych związane z detekcją spoofingu GPS: AV-GPS-Dataset oraz Tuni2025. Zbiory te opisują problem spoofingu na dwóch różnych poziomach. AV-GPS-Dataset zawiera gotowe cechy nawigacyjne i telemetryczne pojazdu, natomiast Tuni2025 udostępnia surowe próbki sygnału radiowego I/Q. Dzięki temu możliwe było porównanie klasycznych modeli uczenia maszynowego działających na cechach tabelarycznych z modelem CNN uczonym bezpośrednio na sygnale.

6.1. AV-GPS-Dataset

AV-GPS-Dataset okazał się odpowiedni dla klasycznych metod uczenia maszynowego, takich jak XGBoost i Random Forest. Dane miały postać tabelaryczną i zawierały cechy opisujące ruch pojazdu, jakość sygnału GPS, liczbę satelitów, błąd śledzenia trajektorii oraz parametry sterowania. Kluczowym etapem przygotowania danych było usunięcie redundantnych kolumn, dodanie cech różnicowych i kroczących oraz wykonanie podziału danych na poziomie całych pomiarów, a nie pojedynczych wierszy. Najważniejszą zaletą tego zbioru była łatwość użycia w klasycznych modelach. Dane nie wymagały przetwarzania surowego sygnału radiowego, dlatego trening modeli był szybki i dobrze interpretowalny. Jednocześnie taki zbiór opisuje spoofing dopiero na poziomie skutków widocznych w danych nawigacyjnych i telemetrycznych, a nie na poziomie fizycznej warstwy sygnału.

6.2. Tuni2025

Tuni2025 był bardziej wymagającym, ale też bardziej wartościowym zbiorem z punktu widzenia detekcji spoofingu na niskim poziomie. Dataset zawierał surowe próbki I/Q, co pozwoliło trenować model CNN bezpośrednio na sygnale oraz przygotować cechy statystyczne dla modeli XGBoost i Random Forest. Dzięki temu możliwe było porównanie podejścia opartego na ręcznie wyznaczonych cechach z podejściem uczącym się reprezentacji bezpośrednio z sygnału. Największymi problemami praktycznymi w Tuni2025 były duży rozmiar plików .bin, identyczność scenariuszy C5 i C7 oraz konieczność poprawnego etykietowania scenariusza SS33, w którym spoofing występuje tylko w określonym przedziale czasowym. Po rozwiązaniu tych problemów

uzyskano kontrolowany pipeline, w którym SS33 pełnił rolę scenariusza testowego niewidzianego podczas treningu. Wyniki pokazały, że klasyczne modele na cechach I/Q są bardzo silnym baseline'em. XGBoost osiągnął najlepszy wynik ogólny, najwyższą dokładność i najniższy FPR, natomiast Random Forest zapewnił dobrą interpretowalność decyzji przez analizę ważności cech. CNN osiągnął nieco niższą dokładność, ale działał bezpośrednio na surowym sygnale I/Q i nie wymagał ręcznej ekstrakcji cech. Jest to istotna zaleta w kontekście potencjalnego wdrożenia.

6.3. Wnioski końcowe

Porównanie obu zbiorów danych pokazuje, że detekcję spoofingu GPS można prowadzić na różnych poziomach przetwarzania. AV-GPS-Dataset pozwala wykrywać skutki spoofingu widoczne w zachowaniu pojazdu i parametrach nawigacyjnych. Tuni2025 pozwala natomiast analizować spoofing bezpośrednio w sygnale radiowym, zanim zostanie on przetworzony przez klasyczny tor odbiornika GNSS. Najlepszy wynik liczbowy uzyskał XGBoost na cechach wyekstrahowanych z okien I/Q, osiągając wysoką dokładność i niski poziom fałszywych alarmów. Random Forest potwierdził, że duża część informacji o spoofingu znajduje się w prostych statystykach amplitudy i mocy sygnału. CNN był najbardziej perspektywicznym podejściem dla pracy bezpośrednio na surowych danych, ponieważ po wytrenowaniu może klasyfikować okna sygnału I/Q bez konieczności ręcznego projektowania cech. Ostatecznie można stwierdzić, że modele drzewiaste są bardzo skuteczne i szybkie dla gotowych cech, natomiast CNN jest bardziej naturalnym kierunkiem dla systemu działającego bezpośrednio na surowym sygnale radiowym. Dalszy rozwój powinien obejmować większą liczbę niezależnych scenariuszy clean i spoofingowych, dokładniejsze testy odporności na inne warunki odbioru oraz bardziej zaawansowane architektury sieci neuronowych analizujące dłuższy kontekst czasowy.

7. Napotkane problemy i pomysły na poprawę

7.1. Duży rozmiar plików binarnych

Jednym z głównych problemów praktycznych był bardzo duży rozmiar plików .bin w zbiorze Tuni2025. Pojedynczy scenariusz zajmował około kilkudziesięciu GB, a pełne przetwarzanie wszystkich plików w sposób naiwny wymagałoby przetworzenia ponad 300 GB surowych danych I/Q.

Problem rozwiązano przez próbkowanie okien z każdego scenariusza. Zamiast analizować cały plik sekwencyjnie, wybierano ograniczoną liczbę okien rozmieszczonych równomiernie w czasie. Takie podejście było uzasadnione, ponieważ sąsiednie fragmenty sygnału w obrębie jednego scenariusza są silnie skorelowane i często niosą bardzo podobną informację.

Dzięki temu udało się znacząco ograniczyć czas uczenia oraz zużycie pamięci, bez zauważalnego pogorszenia jakości modelu. W eksperymentach zaobserwowano, że zwiększanie liczby okien powyżej około 1000 na scenariusz nie przynosiło wyraźnej poprawy wyniku.

7.2. Identyczność plików C5 i C7

Podczas kontroli danych wykryto, że pliki C5.bin oraz C7.bin są bajtowo identyczne, mimo że w zbiorze występowały pod różnymi nazwami i były opisane jako osobne scenariusze clean. Problem wykryto podczas analizy możliwych źródeł data leakage, porównując hashe fragmentów plików oraz ich zawartość.

Było to istotne, ponieważ początkowo planowano wykorzystać jeden z tych scenariuszy jako clean w zbiorze treningowym, a drugi jako clean w zbiorze walidacyjnym. Ponieważ pliki okazały się identyczne, taki podział prowadziłby do przecieku informacji między zbiorami i sztucznego zawyżenia jakości modelu.

Aby uniknąć tego problemu, nie potraktowano C5 i C7 jako niezależnych scenariuszy. Zamiast tego pojedyncze nagranie clean zostało podzielone czasowo na rozłączne fragmenty treningowe i walidacyjne, z dodatkowym buforem czasowym między nimi.

7.3. Etykietowanie scenariusza SS33

Kolejnym istotnym problemem było początkowe błędne etykietowanie scenariusza SS33. Na początku traktowano cały plik jako spoofingowy, ponieważ jego nazwa wskazywała na scenariusz ataku. Późniejsza analiza pokazała jednak, że SS33 jest scenariuszem delayed spoofing, w którym spoofing nie występuje przez cały czas nagrania.

Zgodnie z dokumentacją README, w scenariuszu SS33 spoofing rozpoczyna się dopiero około 70 s po rozpoczęciu nagrania, trwa około 80 s i zostaje usunięty około 150 s. Oznacza to, że fragmenty przed aktywacją spoofingu oraz po jego zakończeniu powinny być traktowane jako clean, a nie jako spoof.

Początkowe oznaczenie całego pliku jako spoofingowego powodowało błędne uczenie modelu, ponieważ fragmenty rzeczywiście czystego sygnału były oznaczane jako atak. Problem rozwiązano przez zastosowanie etykiet czasowych dla SS33. Fragmenty clean i spoof zostały oznaczone zgodnie z czasem występowania ataku, a nie tylko na podstawie nazwy pliku. Dodatkowo pominięto niepewne okolice przejść czasowych między stanem clean i spoof.

7.4. Możliwe dalsze usprawnienia

W kolejnych etapach warto rozszerzyć eksperymenty o dodatkowe scenariusze clean, jeżeli będą dostępne, ponieważ obecna liczba niezależnych nagrań clean jest ograniczona. Pozwoliłoby to przeprowadzić bardziej rygorystyczny podział train/validation/test bez konieczności czasowego dzielenia pojedynczego nagrania.

Drugim kierunkiem poprawy jest dalsze ograniczanie liczby false positive, czyli przypadków, w których fragment clean zostaje oznaczony jako spoofing. Można to osiągnąć przez post-processing czasowy, np. wymaganie kilku kolejnych okien sklasyfikowanych jako spoofing przed wygenerowaniem alarmu. Wstępne testy pokazały, że takie podejście może zmniejszyć liczbę fałszywych alarmów bez zwiększania liczby niewykrytych ataków.

Warto również przetestować bardziej rozbudowane architektury CNN, np. głębsze sieci wielowarstwowe, sieci z blokami rezyduálnymi albo konwolucje dylatacyjne. Takie modele mogłyby analizować większy kontekst czasowy i wykrywać bardziej złożone wzorce w sygnale I/Q niż prostsza sieć konwolucyjna. Jest to szczególnie istotne, ponieważ spoofing może objawiać się nie tylko lokalną zmianą amplitudy lub mocy, ale także subtelną strukturą czasową sygnału.

Należy jednak pamiętać, że bardziej złożona sieć wymaga większej liczby niezależnych scenariuszy treningowych. W przeciwnym razie model może łatwo przeuczyć się na charakterystyczne cechy konkretnych nagrań zamiast nauczyć się ogólnych właściwości spoofingu. Dlatego rozwój architektury CNN powinien być połączony z rozszerzeniem zbioru danych oraz dokładniejszą walidacją na niewidzianych scenariuszach.

Warto było by też lekko za-szumiać datasety aby model był bardziej „odporny” na gorsze warunki np przez dodawanie delikatnych przesunięć fazowych etc.